Databricks Asset Bundle Standards

> **Document Owner:** Platform Engineering | **Status:** Approved | **Last Updated:** February 2026

Overview

Databricks Asset Bundles (DABs) are the standard for infrastructure-as-code deployment. This document defines mandatory patterns for job configuration, serverless environments, hierarchical job architecture, deployment modes, identity, permissions, and variables.

Golden Rules

ID	Rule	Severity	WAF Pillar
IN-01	Serverless compute for all jobs	Required	OpEx
IN-02	Environment-level dependencies (not task)	Critical	OpEx
IN-03	Use `notebook_task` (never `python_task`)	Critical	OpEx
IN-04	Use `dbutils.widgets.get()` for parameters	Critical	OpEx
IN-05	Hierarchical job architecture	Required	OpEx
IN-06	Notebooks in exactly ONE atomic job	Critical	OpEx
IN-07	Use `run_job_task` for job references	Critical	OpEx
IN-08	Include `[${bundle.target}]` prefix	Required	OpEx
IN-11	Use `mode: development` / `mode: production` on all targets	Critical	OpEx
IN-12	Set `run_as` to service principal for staging/prod	Critical	Security
IN-13	Define top-level `permissions` on all bundles	Required	Governance

---

Deployment Modes

Every target must declare `mode: development` or `mode: production`. Never deploy without a mode.

Development Mode

```yaml
targets:
  dev:
    mode: development
    default: true
    workspace:
      host: ${var.dev_host}
```

Development mode automatically:

- Prepends `[dev ${workspace.current_user.short_name}]` to resource names
- Tags all jobs and pipelines with `dev`
- Sets DLT pipelines to `development: true`
- Pauses all schedules and triggers
- Enables concurrent runs for faster iteration
- Disables the deployment lock

Production Mode

```
targets:
  prod:
    mode: production
    workspace:
      host: ${var.prod_host}
      root_path: /Shared/.bundle/${bundle.name}/${bundle.target}
    run_as:
      service_principal_name: ${var.prod_sp_id}
```

Production mode automatically:

- Validates DLT pipelines are `development: false`
- Validates Git branch matches target (if configured)
- Requires `run_as` and `permissions` (recommended)
- Prevents cluster override via `--compute-id`

Custom Presets

Override individual mode behaviors when needed:

```
targets:
  staging:
    presets:
      name_prefix: "staging_"
      pipelines_development: false
      trigger_pause_status: PAUSED
      jobs_max_concurrent_runs: 1
      tags:
        environment: staging
```

Run Identity (`run_as`)

Separate the deploy identity from the run identity. Production jobs must run as service principals.

Top-Level `run_as`

```
bundle:
  name: my_pipeline

run_as:
  service_principal_name: ${var.prod_sp_id}
```

Per-Target Override

```
targets:
  dev:
    mode: development
    run_as:
      user_name: ${workspace.current_user.userName}

  staging:
    run_as:
      service_principal_name: ${var.staging_sp_id}

  prod:
    mode: production
    run_as:
      service_principal_name: ${var.prod_sp_id}
```

Per-Job Override

```
resources:
  jobs:
    sensitive_job:
      run_as:
        service_principal_name: ${var.restricted_sp_id}
```

> **Note:** `run_as` is not supported for model serving endpoints.

Permissions

Define who can view, run, and manage bundle resources.

Top-Level Permissions (Recommended)

Apply to all supported resources in the bundle:

```
permissions:
  - group_name: data-engineering
    level: CAN_MANAGE
  - group_name: data-analysts
```

```
      level: CAN_VIEW
    - service_principal_name: ${var.prod_sp_id}
      level: CAN_MANAGE
```

Allowed top-level levels: `CAN_VIEW`, `CAN_MANAGE`, `CAN_RUN`.

Resource-Level Permissions

For fine-grained access on specific resources:

```
resources:
  jobs:
    sensitive_job:
      permissions:
        - group_name: restricted-team
          level: CAN_MANAGE
        - group_name: data-analysts
          level: CAN_VIEW
```

Precedence Order

1. Resource permissions within target
2. Target-level permissions
3. Resource permissions in bundle
4. Top-level bundle permissions

> **Note:** Permissions for a user/group/SP cannot be defined in both the top-level and resource-level mappings.

Variables

Declare custom variables for environment-specific values. Always define defaults.

Variable Declaration

```
variables:
  catalog:
    description: Unity Catalog name
    default: dev_catalog
  schema:
    description: Target schema
    default: default
  sp_id:
    description: Service principal application ID
```

Per-Target Values

```yaml
targets:
  dev:
    variables:
      catalog: dev_catalog
      schema: dev_schema
  prod:
    variables:
      catalog: prod_catalog
      schema: prod_schema
      sp_id: "68ed9cd5-8923-4851-x0c1-c7536c67ff99"
```

Variable Reference

```yaml
# CORRECT
base_parameters:
  catalog: ${var.catalog}

# WRONG: Missing var. prefix
base_parameters:
  catalog: ${catalog}
```

Complex Variables

Use `type: complex` for structured values:

```yaml
variables:
  notification_config:
    type: complex
    default:
      on_failure:
        - team-alerts@company.com
      on_success:
        - team-reports@company.com
```

Variable Lookups

Resolve object IDs by name:

```yaml
variables:
  warehouse_id:
    lookup:
      warehouse: "Shared SQL Warehouse"
```

Precedence Order

1. `--var="key=value"` CLI option
2. `BUNDLE_VAR_` environment variables
3. `.databricks/bundle/<target>/variable-overrides.json`
4. Target-level `variables` mapping
5. Top-level `variables` default

Serverless Job Template

Every job must include this serverless configuration:

```
resources:
  jobs:
    my_job:
      name: "[${bundle.target}] My Job Name"

      environments:
        - environment_key: "default"
          spec:
            environment_version: "4"
            dependencies:
              - "pandas==2.0.3"

      tasks:
        - task_key: task_name
          environment_key: default
          notebook_task:
            notebook_path: ../src/my_script.py
            base_parameters:
              catalog: ${var.catalog}
```

Common Mistakes

```
# WRONG: Task-level libraries (use environments block instead)
tasks:
  - task_key: my_task
    libraries:
      - pypi: { package: pandas }

# WRONG: python_task doesn't work with serverless
tasks:
  - task_key: my_task
    python_task:
      python_file: ../src/script.py
```

Library Dependencies

Environment Block (Serverless)

For serverless jobs, declare dependencies in `environments`:

```
environments:
  - environment_key: "default"
    spec:
      environment_version: "4"
      dependencies:
        - "pandas==2.0.3"
        - "scikit-learn>=1.3.0"
```

Requirements File

Reference a `requirements.txt` for longer dependency lists:

```
tasks:
  - task_key: my_task
    libraries:
      - requirements: ./requirements.txt
```

Python Wheel Files

```
tasks:
  - task_key: my_task
    libraries:
      - whl: ./dist/my_package-0.1.0-py3-none-any.whl
      - whl: /Volumes/main/default/libs/shared-0.2.0.whl
```

PyPI Packages (Classic Compute)

```
tasks:
  - task_key: my_task
    libraries:
      - pypi:
          package: numpy==1.25.2
```

Parameter Passing

In YAML

```yaml
notebook_task:
  notebook_path: ../src/script.py
  base_parameters:
    catalog: ${var.catalog}
    schema: ${var.gold_schema}
```

In Python

```python
# CORRECT
def get_parameters():
    catalog = dbutils.widgets.get("catalog")
    schema = dbutils.widgets.get("schema")
    return catalog, schema

# WRONG: argparse fails in notebook_task
parser = argparse.ArgumentParser()
args = parser.parse_args()  # ERROR!
```

Hierarchical Job Architecture

Each notebook appears in exactly ONE atomic job. Higher levels reference jobs, not notebooks.

Layer 1: Atomic Jobs (Notebook References)

```yaml
resources:
  jobs:
    tvf_deployment_job:
      name: "[${bundle.target}] TVF Deployment"

      tasks:
        - task_key: deploy
          notebook_task:
            notebook_path: ../../src/deploy_tvfs.py

      tags:
        job_level: atomic
```

Layer 2: Composite Jobs (Job References)

```yaml
resources:
  jobs:
    semantic_layer_setup:
      name: "[${bundle.target}] Semantic Layer Setup"
```

```
      tasks:
        - task_key: deploy_tvfs
          run_job_task:
            job_id: ${resources.jobs.tvf_deployment_job.id}

        - task_key: deploy_metrics
          depends_on:
            - task_key: deploy_tvfs
          run_job_task:
            job_id: ${resources.jobs.metric_view_job.id}

      tags:
        job_level: composite
```

Layer 3: Orchestrators (References Composites)

```
resources:
  jobs:
    master_setup:
      name: "[${bundle.target}] Master Setup"

      tasks:
        - task_key: gold_setup
          run_job_task:
            job_id: ${resources.jobs.gold_setup_job.id}

        - task_key: semantic_setup
          depends_on:
            - task_key: gold_setup
          run_job_task:
            job_id: ${resources.jobs.semantic_layer_setup.id}

      tags:
        job_level: orchestrator
```

DLT Pipeline Configuration

```
resources:
  pipelines:
    silver_pipeline:
      name: "[${bundle.target}] Silver Pipeline"

      root_path: ../src/silver_pipeline
      catalog: ${var.catalog}
      schema: ${var.silver_schema}

      serverless: true
      photon: true
```

```
        channel: CURRENT
        edition: ADVANCED

        libraries:
          - notebook:
              path: ../src/silver_pipeline/transforms.py
```

Triggering from Workflows

```
tasks:
  - task_key: run_pipeline
    pipeline_task:
      pipeline_id: ${resources.pipelines.silver_pipeline.id}
      full_refresh: false
```

Supported Resource Types

Resource	Description
`jobs`	Jobs and tasks
`pipelines`	Lakeflow Spark Declarative Pipelines (DLT)
`schemas`	Unity Catalog schemas
`volumes`	Unity Catalog volumes
`clusters`	Compute clusters
`catalogs`	Unity Catalog catalogs
`dashboards`	AI/BI dashboards (Lakeview)
`registered_models`	Unity Catalog registered models
`model_serving_endpoints`	Model serving endpoints
`experiments`	MLflow experiments
`apps`	Databricks Apps
`sql_warehouses`	SQL warehouses
`secret_scopes`	Secret scopes
`quality_monitors`	Data quality monitors

Reference resources using `${resources.<type>.<name>.<field>}`:

```
existing_cluster_id: ${resources.clusters.my_cluster.id}
pipeline_id: ${resources.pipelines.my_pipeline.id}
```

Complete Bundle Template

```yaml
bundle:
  name: my_data_pipeline

variables:
  catalog:
    description: Unity Catalog name
    default: dev_catalog
  schema:
    description: Target schema
    default: default

permissions:
  - group_name: data-engineering
    level: CAN_MANAGE
  - group_name: data-analysts
    level: CAN_VIEW

run_as:
  service_principal_name: ${var.sp_id}

targets:
  dev:
    mode: development
    default: true
    workspace:
      host: ${var.dev_host}
    run_as:
      user_name: ${workspace.current_user.userName}
    variables:
      catalog: dev_catalog

  prod:
    mode: production
    workspace:
      host: ${var.prod_host}
      root_path: /Shared/.bundle/${bundle.name}/${bundle.target}
    variables:
      catalog: prod_catalog
      sp_id: "68ed9cd5-8923-4851-x0c1-c7536c67ff99"

resources:
  jobs:
    transform_job:
      name: "[${bundle.target}] Transform Pipeline"

      environments:
        - environment_key: "default"
          spec:
```

```yaml
            environment_version: "4"
            dependencies:
              - "pandas==2.0.3"

      tags:
        team: data-engineering
        cost_center: CC-1234
        environment: ${bundle.target}

      tasks:
        - task_key: transform
          environment_key: default
          notebook_task:
            notebook_path: ../src/transform.py
            base_parameters:
              catalog: ${var.catalog}
              schema: ${var.schema}
```

Path Resolution

YAML Location	Path to `src/`
`resources/*.yml`	`../src/`
`resources/<layer>/*.yml`	`../../src/`

Directory Structure

```
my_bundle/
├── databricks.yml                # Bundle configuration
├── requirements.txt              # Shared dependencies
├── src/                          # Source code
│   ├── transform.py
│   └── silver_pipeline/
│       └── transforms.py
└── resources/                    # Resource YAML files
    ├── orchestrators/            # Layer 3
    │   └── master_setup.yml
    ├── semantic/                 # Domain
    │   ├── semantic_setup.yml    # Layer 2
    │   └── tvf_job.yml           # Layer 1
    ├── monitoring/               # Domain
    │   └── lakehouse_job.yml     # Layer 1
    └── pipelines/
        └── silver_pipeline.yml
```

Validation Checklist

Deployment & Identity

- [ ] `mode: development` on dev target
- [ ] `mode: production` on prod target
- [ ] `run_as` set to service principal for staging/prod
- [ ] Top-level `permissions` block defined
- [ ] `variables` declared with defaults

Job Configuration

- [ ] `environments:` block at job level
- [ ] `environment_key: default` on every task
- [ ] `[${bundle.target}]` prefix in job name
- [ ] `notebook_task` (never `python_task`)
- [ ] `base_parameters` dictionary format
- [ ] `${var.name}` for variable references
- [ ] Each notebook in ONE atomic job only
- [ ] Orchestrators use `run_job_task` only

Tags

- [ ] `team`, `cost_center`, `environment` tags on all jobs

Common Errors

Error	Cause	Solution
`arguments are required`	argparse	Use `dbutils.widgets.get()`
`Invalid task type`	python_task	Use `notebook_task`
`Variable not found`	Missing prefix	Use `${var.name}`
`run_as not supported`	model_serving_endpoint	Remove `run_as` from that resource
`Permission overlap`	Same identity in top-level + resource	Use one or the other, not both
`Git branch mismatch`	Production mode branch check	Deploy from correct branch or use `--force`

CI/CD Best Practices

Core Principles

Principle	Description
Version control everything	All code, configs, infra in Git

Principle	Description
Automate testing	Unit, integration, data quality tests
Use IaC	Asset Bundles for all resources
Environment isolation	Separate dev/staging/prod
Unified asset management	DABs for code + infra together

Branching Strategy

```
main (production)
  |
  +-- staging
        |
        +-- feature/XXX-description
```

Branch	Deploys To	Triggered By
`main`	Production	Merge from staging
`staging`	Staging	Merge from feature
`feature/*`	Development	Push

CI Workflow Template

```yaml
# .github/workflows/ci.yml
name: Databricks CI

on:
  push:
    branches: [main, staging]
  pull_request:
    branches: [main, staging]

jobs:
  validate:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Install Databricks CLI
        run: pip install databricks-cli

      - name: Validate Bundle
        run: databricks bundle validate
        env:
          DATABRICKS_HOST: ${{ secrets.DATABRICKS_HOST }}
          DATABRICKS_TOKEN: ${{ secrets.DATABRICKS_TOKEN }}
```

```yaml
  deploy-staging:
    if: github.ref == 'refs/heads/staging'
    needs: validate
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Deploy to Staging
        run: databricks bundle deploy -t staging
        env:
          DATABRICKS_HOST: ${{ secrets.DATABRICKS_HOST }}
          DATABRICKS_TOKEN: ${{ secrets.DATABRICKS_TOKEN }}

  deploy-prod:
    if: github.ref == 'refs/heads/main'
    needs: validate
    runs-on: ubuntu-latest
    environment: production
    steps:
      - uses: actions/checkout@v4

      - name: Deploy to Production
        run: databricks bundle deploy -t prod
        env:
          DATABRICKS_HOST: ${{ secrets.DATABRICKS_HOST }}
          DATABRICKS_TOKEN: ${{ secrets.DATABRICKS_TOKEN }}
```

Best Practices

Practice	Implementation
Workload identity federation	Use OIDC, not static tokens
Environment promotion	dev -> staging -> prod
Automated testing	pytest for transforms, DLT for quality
Rollback procedures	Document and test regularly
Secrets management	GitHub Secrets, never in code

Related Documents

- Python Development
- Serverless Compute
- Reliability & DR
- UC Governance

References

- Asset Bundles
- Deployment Modes
- Run Identity
- Variables
- Bundle Resources
- Permissions
- Library Dependencies
- CI/CD Best Practices
- MLOps Stacks